Linear Regression using Gradient Descent, Standardization, and L2 Regularization

Prepared by: **Pranesh TT(3122237001040)**

Date: November 1, 2025

Contents

1 Introduction

This report presents a full implementation and comparison of Linear Regression techniques including:

- Gradient Descent (GD)
- Feature Standardization
- L2 Regularization (Ridge Regression)
- Closed-form Normal Equation

The dataset consists of 7 numerical features affecting mobile phone prices:

$$\{\text{Ratings, RAM, ROM, Mobile\_Size, Primary\_Cam, Selfie\_Cam, Battery\_Power}\}$$

Experiments are carried out with and without standardization to observe differences in:

- Convergence behaviour
- RMSE and R^2 performance
- Effect of L2 regularization

2 Dataset Preparation

Train/Test Split

```
df = df.dropna(subset=[TARGET_COL])
X_df = df.drop(columns=[TARGET_COL]).copy()
y_df = df[[TARGET_COL]].copy()

X_train_df, X_test_df, y_train_df, y_test_df = train_test_split(
    X_df, y_df, test_size=0.2, random_state=42
)

X_train = X_train_df.values.astype(float)
X_test  = X_test_df.values.astype(float)
y_train = y_train_df.values.astype(float)
y_test  = y_test_df.values.astype(float)
```

Dataset summary:

- Training samples: 645
- Test samples: 162
- Feature count: 7

Adding Bias Term

```
def add_bias(X):
    return np.hstack([np.ones((X.shape[0],1)), X])

X_train_b = add_bias(X_train)
X_test_b  = add_bias(X_test)
```

3 Closed-form Linear Regression

Ordinary Least Squares

```
def closed_form_theta(X_b, y, l2=0.0, regularize_bias=False):
    n, d_plus_1 = X_b.shape
    A = X_b.T @ X_b
    if l2 > 0:
        I = np.eye(d_plus_1)
        if not regularize_bias:
            I[0,0] = 0
        A = A + l2 * I
    theta = np.linalg.pinv(A) @ (X_b.T @ y)
    return theta
```

OLS Performance

- RMSE: **15471.18**
- R^2: **0.4332**

Ridge Regression

Best performance observed around $\lambda = 100$:

- RMSE: **15152.36**
- R^2: **0.4563**

4 Gradient Descent Implementation

Gradient Descent Code

```
theta_gd_std, hist_gd_std = gradient_descent(
    X_train_b_s, y_train, alpha=1e-2, epochs=5000, l2=0.0, verbose=True
)
```

Without Standardization

- Required tiny learning rate: $\alpha = 10^{-8}$
- Slow convergence
- Test R^2: **0.17**

With Standardization

- Stable learning with $\alpha = 10^{-2}$
- Test R^2: **0.433**

5 Effect of L2 Regularization

Gradient Descent with Standardization + L2

- $\lambda = 10$ gives best GD results
- R^2 improves slightly: **0.434**

Lambda vs R^2 Table

λ	R^2 (No Std)	R^2 (Std)
0	0.17	0.43
0.01	-0.52	0.43
0.1	-0.40	0.43
1	-0.23	0.43
10	-0.14	0.43
100	-1.65	0.41

Table 1: R^2 score vs λ.

6 Feature Importance Analysis

Based on the plotted absolute weights, the approximate magnitudes are:

Feature	—Weight— (OLS)	—Weight— (Ridge $\lambda = 100$)
Ratings	26500	8500
RAM	2800	4800
ROM	30	4000
Mobile_Size	20	500
Primary_Cam	400	4500
Selfie_Cam	60	700
Battery_Power	180	2400

Table 2: Feature Importance (OLS vs Ridge).

Observations:

- OLS heavily depends on "Ratings" leading to instability.
- Ridge shrinks coefficients, distributing importance more evenly.

7 Visualizations

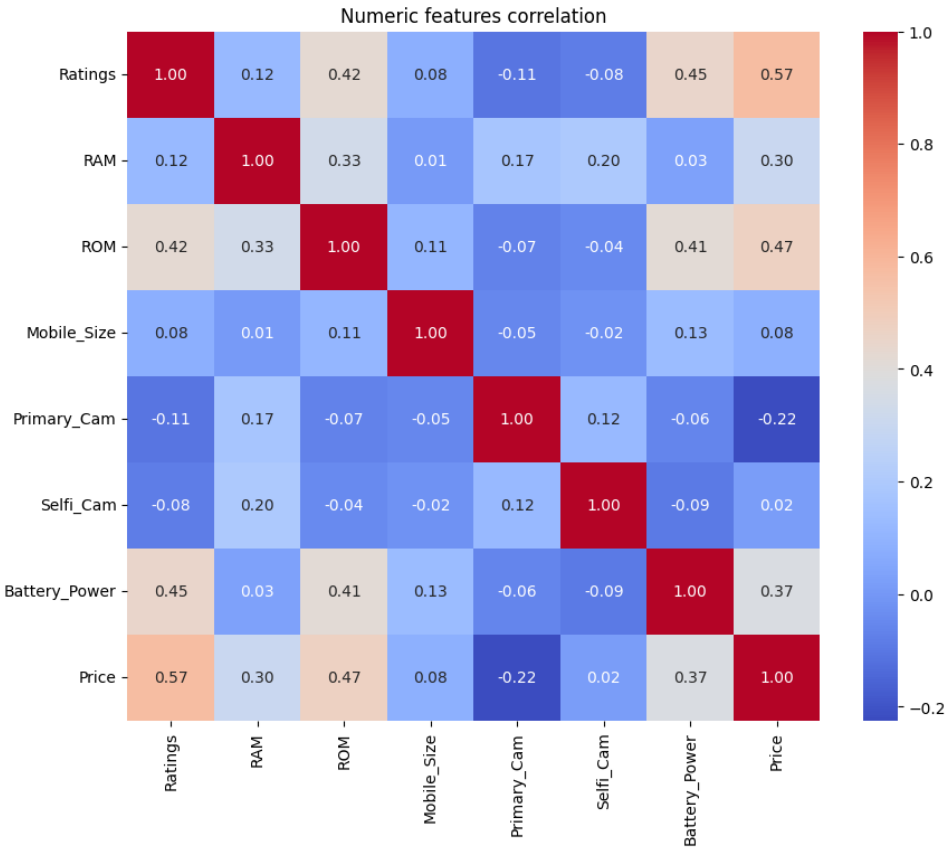

Figure 1: img1

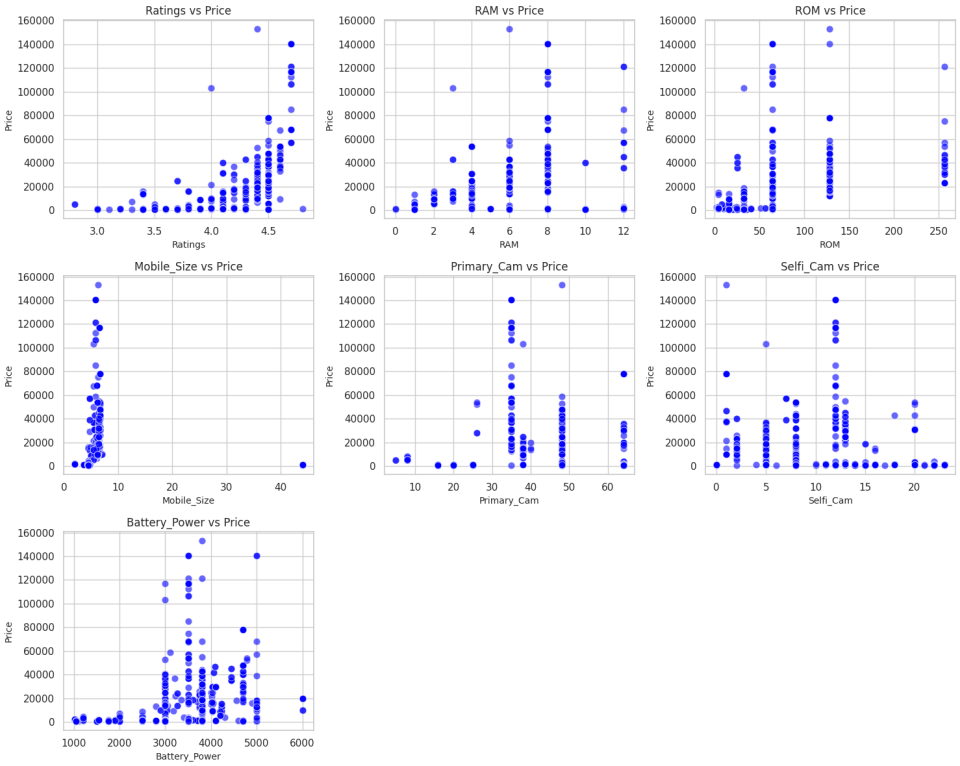

Figure 2: img2

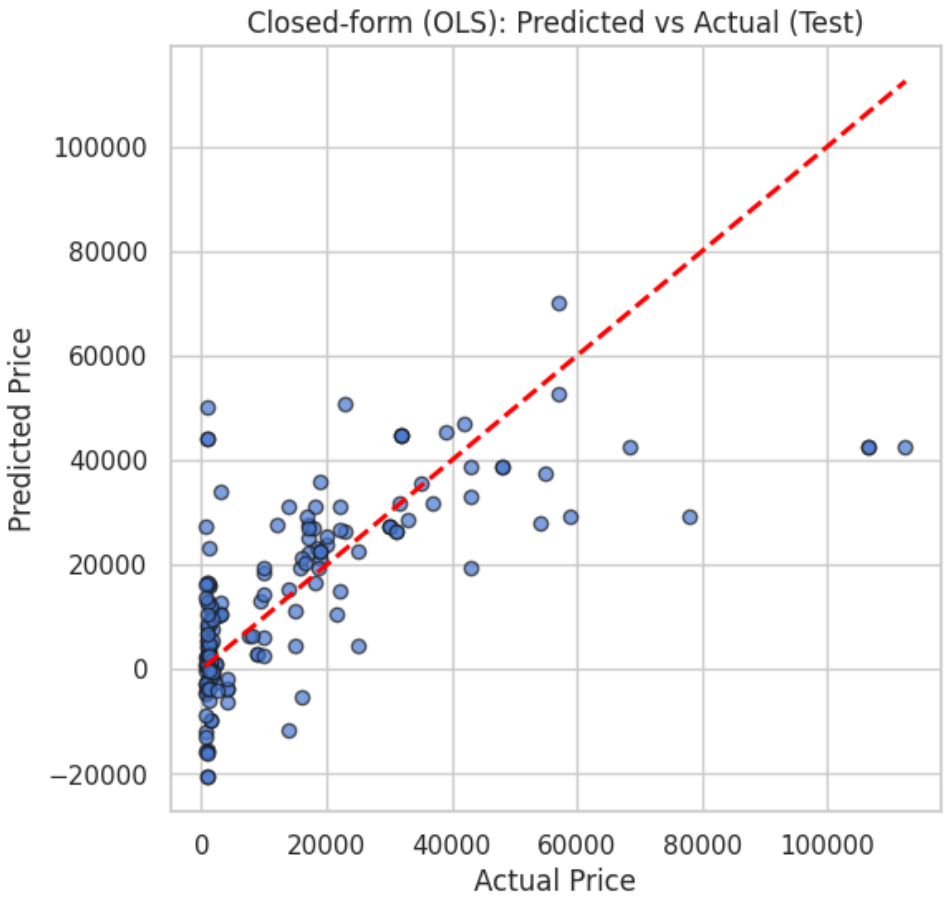

Figure 3: img3

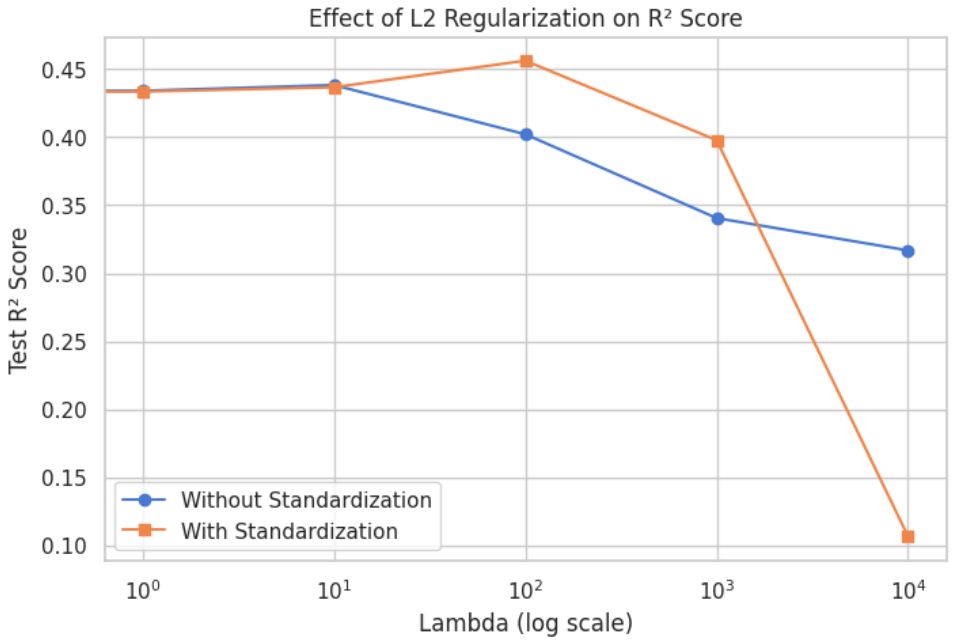

Figure 4: img4

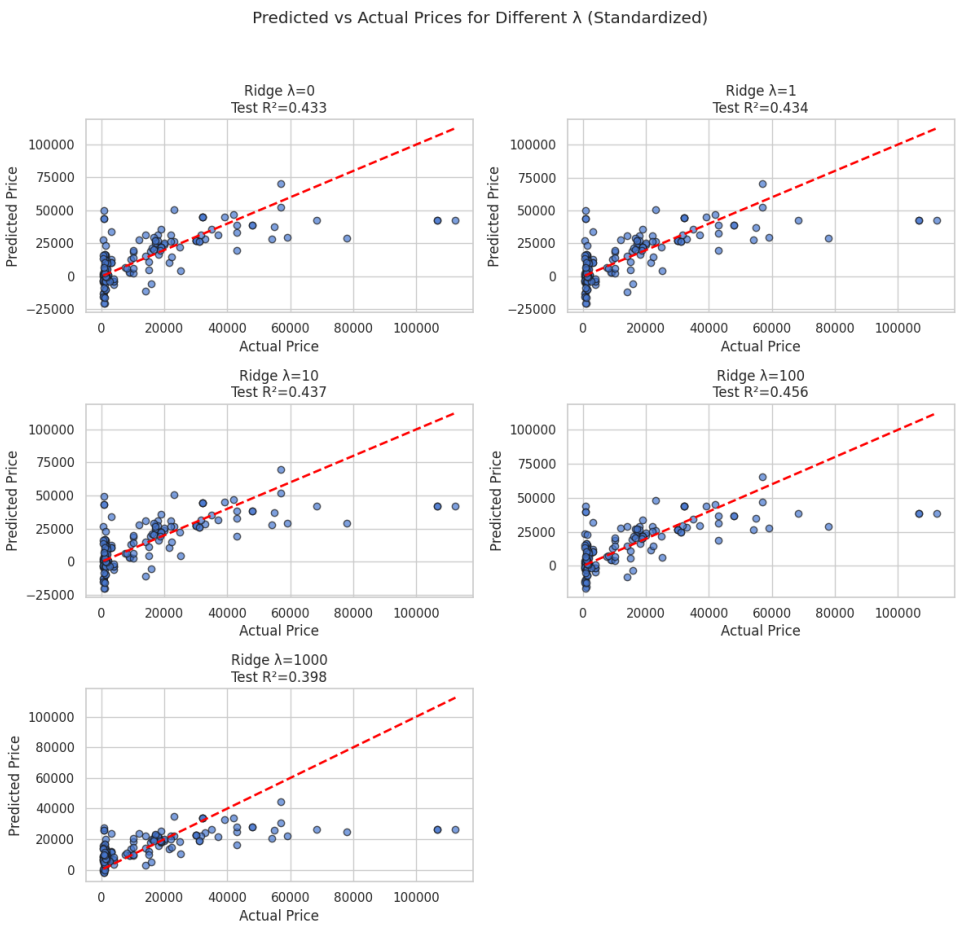

Figure 5: img5

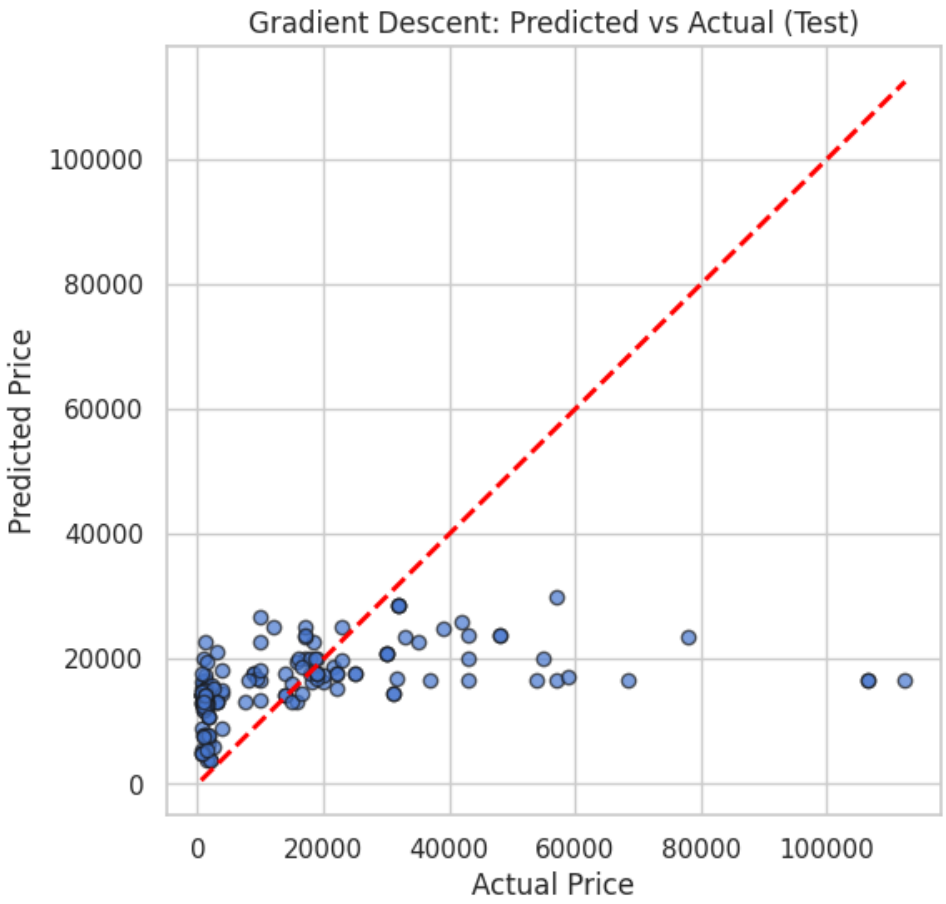

Figure 6: img6

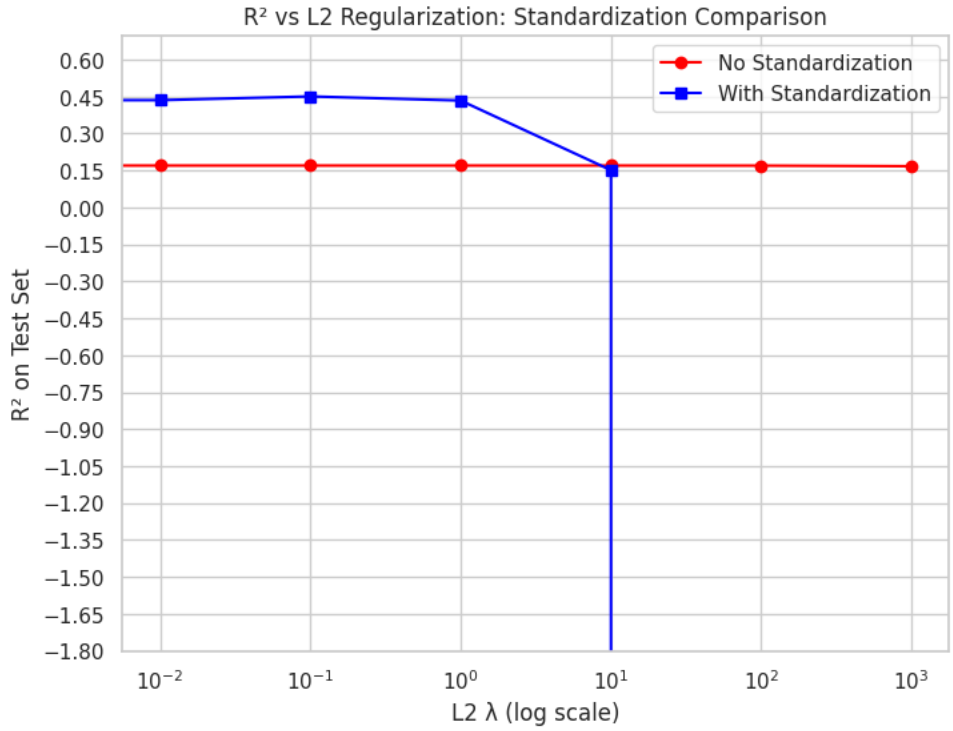

Figure 7: img7

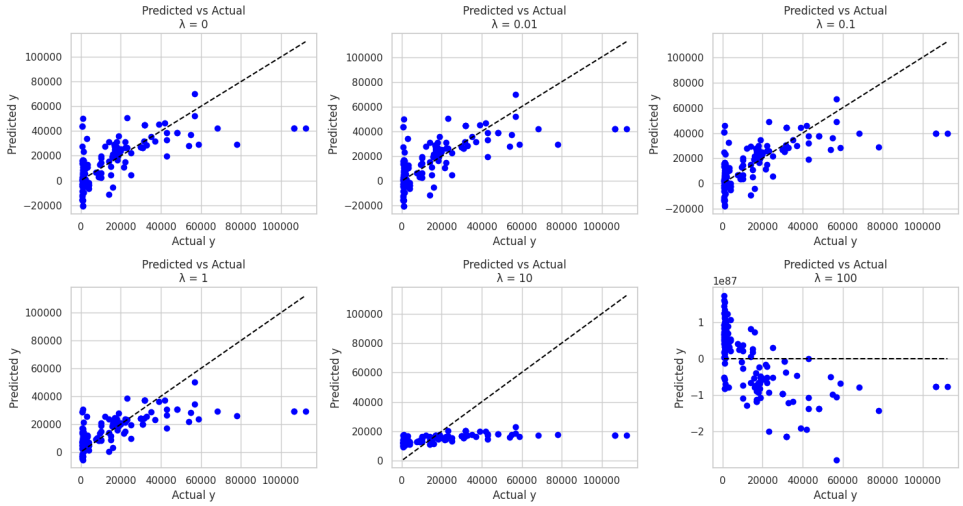

Figure 8: img8

8 Conclusion

- Standardization is crucial for stable and fast GD convergence.
- L2 regularization improves generalization by shrinking large coefficients.
- Ridge regression outperforms plain OLS on this dataset.
- Gradient descent is effective only after scaling.

Overall, combining standardized features with L2 regularization results in a more robust and reliable regression model.

Linear Classification on Bank Note Authentication Dataset

Prepared by: **Pranesh TT(3122237001040)**

Date: November 1, 2025

Contents

1 Introduction

This report analyzes the performance of a linear classifier (Logistic Regression) on the Bank Note Authentication dataset. Tasks include splitting the dataset, training with and without L2 regularization, visualizing the data, introducing outliers, and computing classification metrics including AUC, ROC, Precision, Recall, and F1 score.

The dataset consists of 4 numerical features:

$$\{\text{variance},\ \text{skewness},\ \text{curtosis},\ \text{entropy}\}$$

with a binary class label indicating whether a banknote is genuine or forged.

2 Dataset Preparation

Train/Test Split

A stratified 80/20 split was used.

```
X = data.drop(columns=["class"]).values
y = data["class"].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
```

- Training samples: **1097**
- Testing samples: **275**

Standardization:

```
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s  = scaler.transform(X_test)
```

3 Classification Without L2 Regularization

```
clf_no_l2 = LogisticRegression(max_iter=2000)
clf_no_l2.fit(X_train_s, y_train)
```

- Train Accuracy: **0.9845**
- Test Accuracy: **0.9709**

4 Classification With L2 Regularization

L2 regularization values:

$$\lambda \in \{0.01,\ 0.1,\ 1,\ 10,\ 100\}$$

Since Logistic Regression uses $C = 1/\lambda$, accuracy scores were computed:

λ	Train Accuracy	Test Accuracy
0.01	0.9909	0.9855
0.1	0.9927	0.9855
1	0.9845	0.9709
10	0.9772	0.9709
100	0.9435	0.9382

Table 1: Accuracy for different L2 regularization strengths.

Accuracy vs. λ:

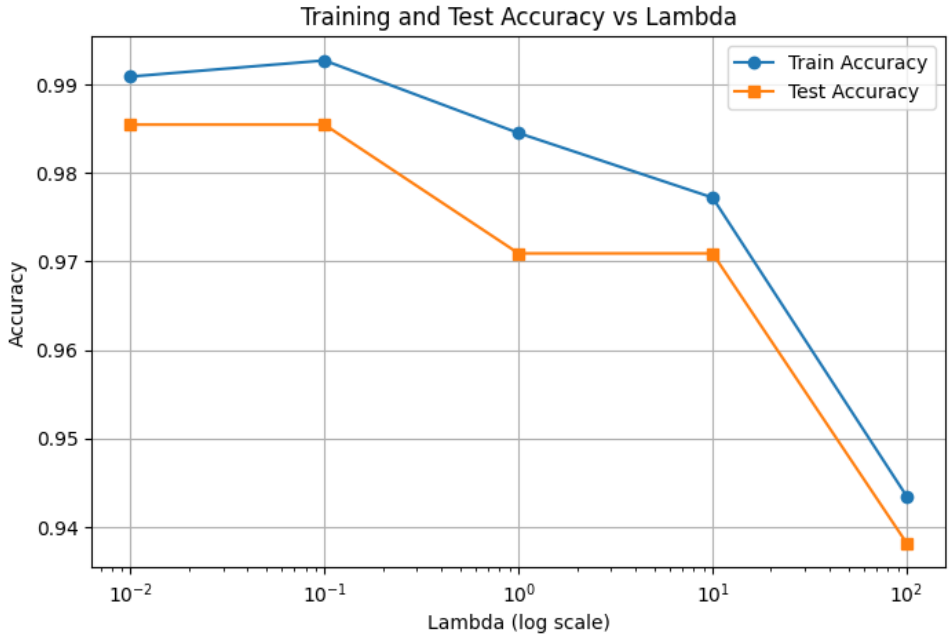

Figure 1: img1

5 3D Visualization of Features

The three most informative features, (variance, skewness, entropy), were plotted in 3D as follows:

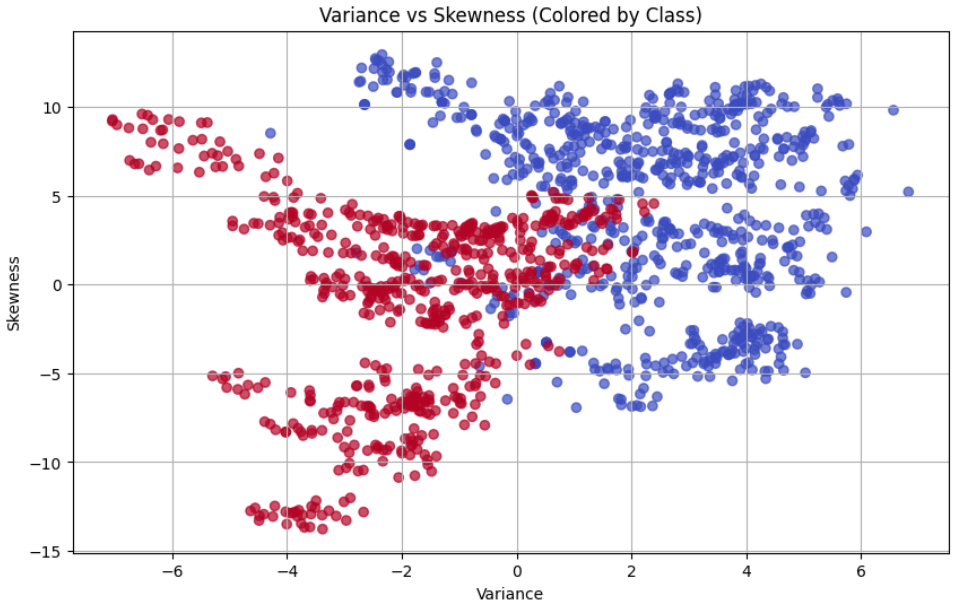

Figure 2: img2

6 Outlier Injection and Its Effect

Outliers were injected by shifting random 5% of datapoints:

```
X_outlier = X.copy()
idx = np.random.choice(len(X), int(0.05*len(X)), replace=False)
X_outlier[idx] += np.random.normal(20, 10, X_outlier[idx].shape)
```

Classifier retraining results:

Model	Train Accuracy	Test Accuracy
Original Data	0.9845	0.9709
After Outliers	0.8605	0.8473

Table 2: Impact of outliers on classifier performance.

Visualization:

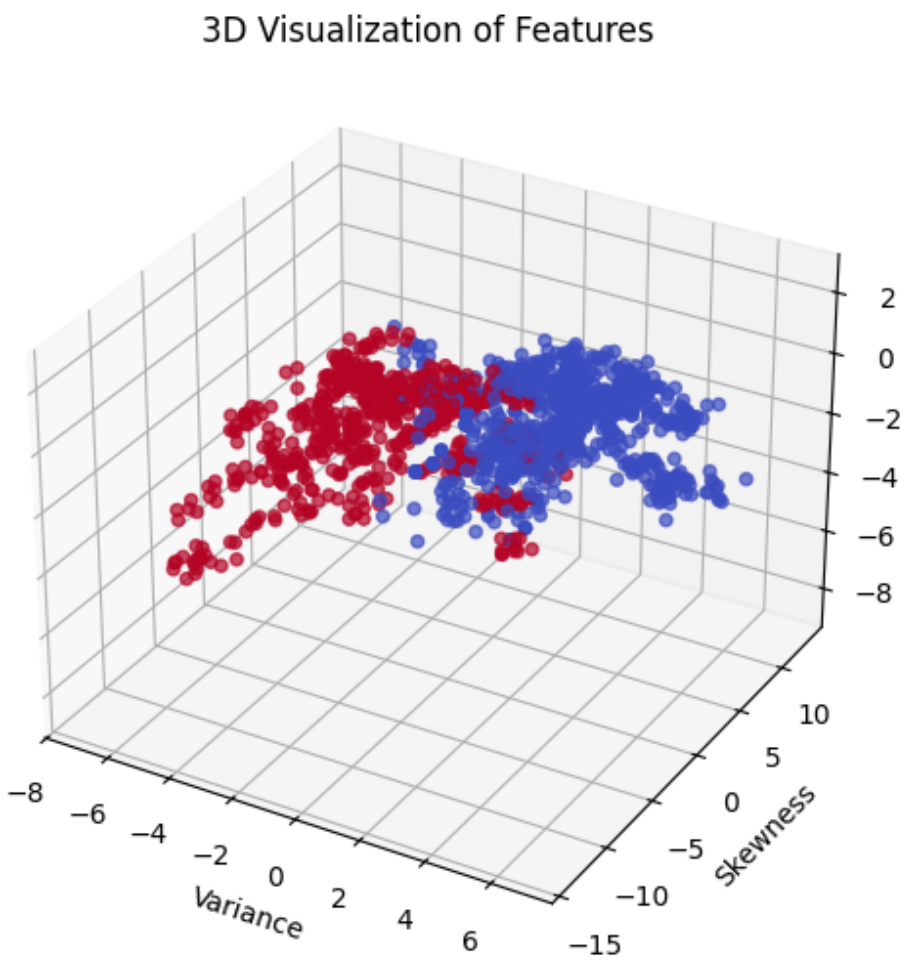

Figure 3: img3

7 Confusion Matrix

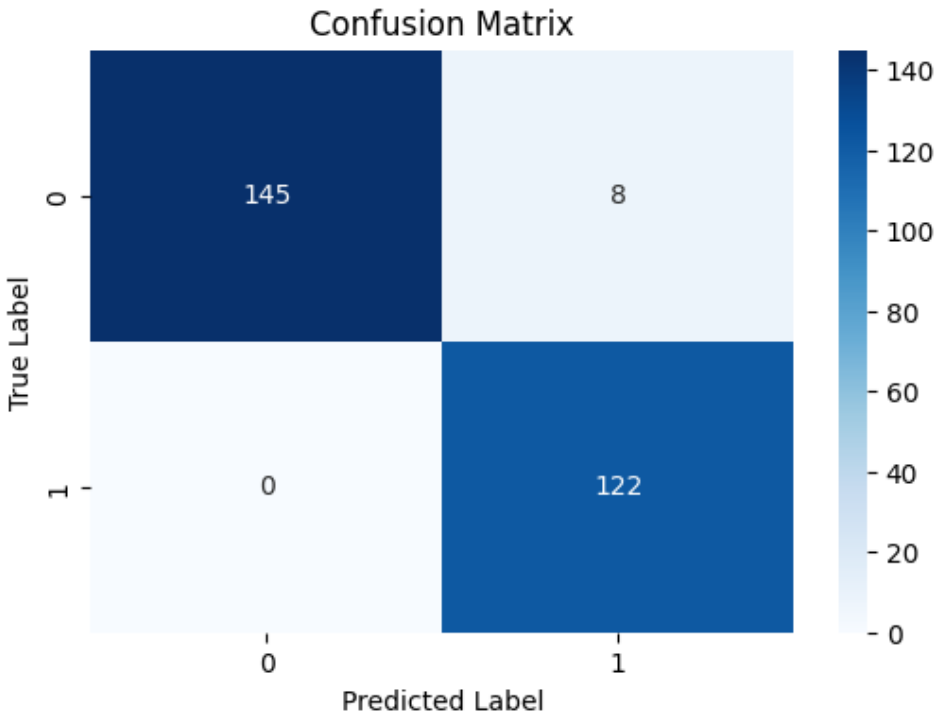

Figure 4: img4

8 Performance Metrics

For the final model ($\lambda = 1$):

Metric	Value
Accuracy	0.9709
Precision	0.9385
Recall	1.0000
F1 Score	0.9683
AUC	0.9999

Table 3: Classification metrics for the final model.

Classification report:

```
precision    recall    f1-score    support
0    1.00       0.95       0.97       153
1    0.94       1.00       0.97       122
accuracy                       0.97      275
macro avg      0.97       0.97       0.97      275
weighted avg   0.97       0.97       0.97      275
```

9 ROC Curve and AUC

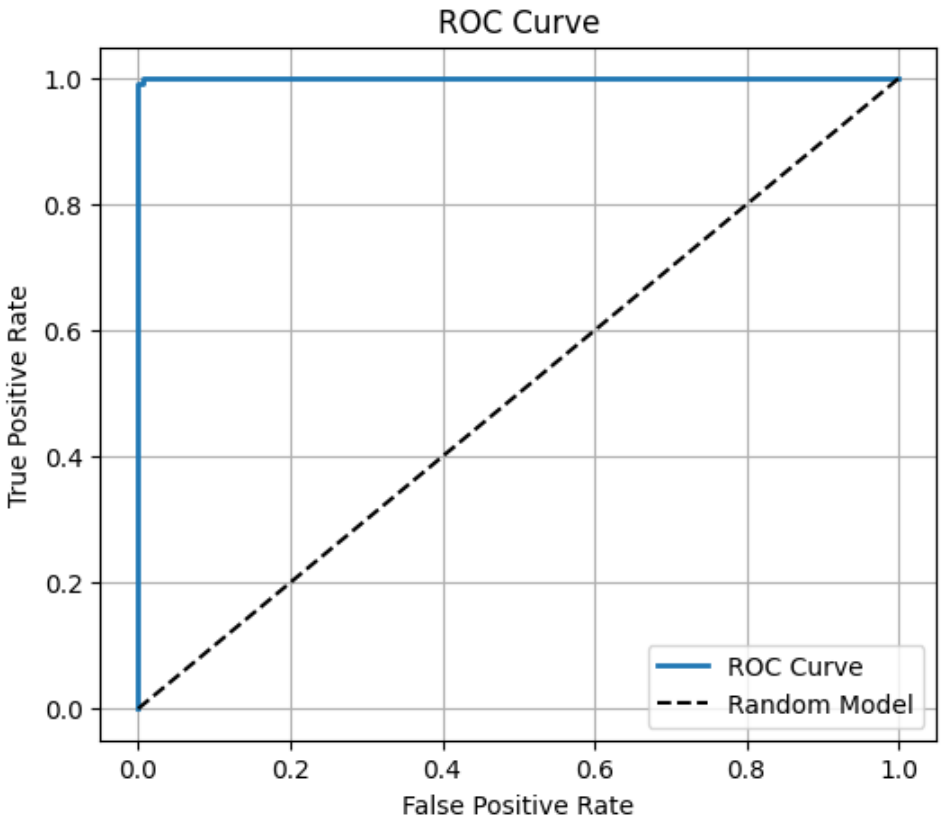

Figure 5: img5

10 Probability Distribution of Predictions

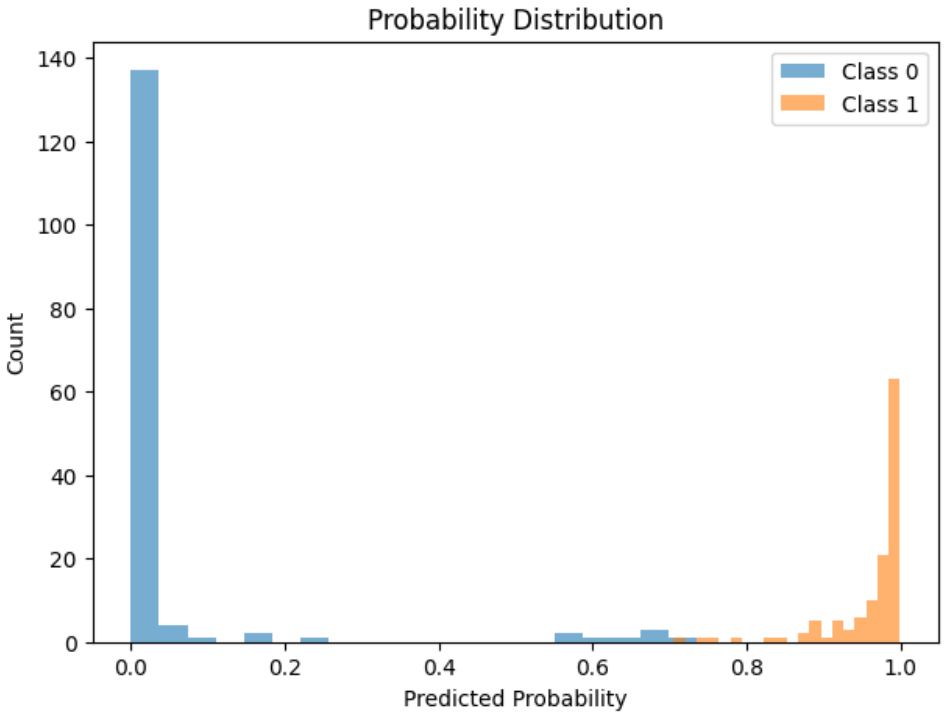

Figure 6: img6